Cervid Ballistics — Mathematical Models and Design Methodology

Technical Reference

August 2026

Introduction

Cervid Ballistics is a single-file, offline-capable exterior ballistics calculator covering air rifle, rimfire, and centrefire disciplines. This document describes the mathematical models underpinning the trajectory engine and the engineering methodology that shaped the application's design over its development history. It is intended as a technical reference for anyone reviewing, extending, or auditing the calculator's physics or its software design decisions.

Two caveats apply throughout. First, several of the models below are widely used, well-established *approximations* rather than first-principles derivations — Miller's stability formula and Litz's spin-drift formula are the clearest examples — and the application's own code comments are deliberately explicit about where this is the case. Second, all internal computation is carried out in imperial units (feet, seconds, grains, degrees Fahrenheit-derived Kelvin) regardless of the unit system the user has selected for display; metric values are converted only at the input and output boundary. This choice is discussed further in the design methodology section.

Part 1 — Mathematical Models

1. Coordinate System and Numerical Integration

The trajectory engine integrates the equations of motion using a fourth-order Runge–Kutta (RK4) method, but — unlike most simple ballistic solvers, which step forward in time — it integrates with **horizontal distance travelled (x, in feet) as the**

independent variable, not time. A fixed step of 3 feet (exactly one yard) is used throughout.

The state vector carried through the integration is:

- **y** — vertical position relative to the bore line (feet)
- **v_x, v_y** — horizontal and vertical velocity components (ft/s)
- **t** — elapsed time (seconds)

and the derivatives with respect to x are:

$$dy/dx = v_y / v_x$$
$$dv_x/dx = -D / v$$
$$dv_y/dx = (-D \cdot (v_y/v) - g_{eff}) / v_x$$
$$dt/dx = 1 / v_x$$

where $v = \sqrt{(v_x^2 + v_y^2)}$ is the speed along the flight path, D is the drag deceleration (Section 2), and g_{eff} is gravitational acceleration after the Eötvös correction (Section 8).

Why integrate in x rather than t? Stepping in fixed 1-yard increments guarantees that every row of the range card — whichever interval the user has configured, in yards or metres — lands on an exact, physically computed sample point rather than an interpolated one. A time-stepped integrator would need to interpolate between two unevenly-spaced samples to recover the state at a clean range mark, introducing a small but avoidable extra source of error. It also made it straightforward to later add a "dense" sampling mode (one recorded point per yard, regardless of the table's own interval) purely for smooth chart rendering, without touching the underlying physics at all — the same integration loop just records more of the points it was already computing.

The classical RK4 update, applied at each step, is:

$$k1 = f(state)$$
$$k2 = f(state + (dx/2) \cdot k1)$$
$$k3 = f(state + (dx/2) \cdot k2)$$
$$k4 = f(state + dx \cdot k3)$$
$$state_{next} = state + (dx/6) \cdot (k1 + 2k2 + 2k3 + k4)$$

applied simultaneously across all four state variables (y, v_x, v_y, t).

2. Aerodynamic Drag

Deceleration along the flight path is computed from the standard point-mass exterior-ballistics drag equation:

$$D = \rho_{ratio} \times K \times Cd(Mach) \times v^2 / BC$$

where ρ_{ratio} is the air density relative to the standard reference atmosphere (Section 6), $K = 2.085 \times 10^{-4}$ is a fixed empirical constant, $Cd(Mach)$ is the drag coefficient at the current Mach number, v is true airspeed, and BC is the projectile's ballistic coefficient (in the same drag-model standard, G1/G7/GA, as the Cd table being used). Because BC already normalises for the reference projectile's own geometry under whichever

standard is selected, the equation needs no separate diameter, mass, or cross-sectional-area terms.

Cd(Mach) is obtained by linear interpolation between adjacent points of a fixed reference table, selected according to the drag model the user has chosen:

Model	Reference shape	Notes
G1	Flat-base, blunt "generic" projectile	The oldest and most widely published standard; most manufacturer-quoted BCs are G1
G7	Boat-tail, streamlined long-range profile	A closer match to modern low-drag rifle bullets; produces a flatter, more realistic transonic drag curve for those shapes
GA	Diabolo airgun pellet	A community-sourced table (matching the data embedded in ChairGun/Strelok Pro) built specifically because G1 and G7 — both shaped around rifle-bullet geometry — fit diabolo pellets poorly, particularly the sharp transonic drag rise pellets show above roughly Mach 0.8. Treated as the best available reference rather than an authoritative published standard

Each table stores [Mach, Cd] pairs; values below the lowest or above the highest tabulated Mach number are clamped to the nearest endpoint rather than extrapolated.

3. Ballistic Coefficient Standards (G1 ↔ G7)

A G1 BC and a G7 BC for the same physical projectile are **not the same number** — each normalises drag against a differently shaped reference standard, so the two are not interchangeable. The application always requires a G1 BC. A G7 BC is optional: if the user supplies a genuine published G7 figure, it is used directly whenever the G7 drag model is active. If it is left blank, the model falls back to an estimate:

$$BC(G7,\ \text{estimated}) = BC(G1) \,/\, 1.9$$

The ratio 1.9 is a commonly cited approximation for modern boat-tail rifle bullets; it has no general validity for pellets or flat-base projectiles, and the interface flags the value as an estimate whenever it is in use rather than presenting it as an entered figure.

4. Zeroing (Sight-In) Solver

Before the visible trajectory is computed, the engine has to solve for the vertical launch angle (relative to the sight line) that makes the projectile cross the sight line exactly at the user's chosen zero distance — the same problem a shooter solves by adjusting turrets at the range.

This is done in two stages:

1. **Time-to-zero pass.** A short simulation at zero launch angle establishes t_0, the time of flight to the zero distance.

2. **Iterative angle refinement.** An initial vertical velocity offset is estimated analytically —

$$v_{yB} = (\text{scopeHeight}/12 + \text{zeroDistance}\cdot\tan(\text{angle}) + \tfrac{1}{2}\cdot g\cdot t_0^2) / t_0 - MV\cdot\sin(\text{angle})$$

— then refined by up to eight passes of a damped correction loop: each pass re-simulates the shot with the current candidate offset, measures the vertical gap between point of impact and true sight line at the zero distance, and nudges the offset by 85% of the gap (scaled by time) before trying again. The loop exits early once the residual gap is under 0.005 inch. This is a simplified damped fixed-point solver, not a formal bisection or Newton–Raphson method — it converges quickly in practice because the relationship between launch angle and point of impact is close to linear over the small angles involved.

5. Wind Deflection

Wind input uses a clock-face convention, matching how a Kestrel or any other wind meter is read with the vane-into-the-wind technique: point it at the target (12 o'clock), turn it until the impeller stalls, and read whichever hour the back of the meter faces. 3 o'clock means a full-value crosswind FROM the shooter's right, 9 o'clock a full-value crosswind FROM the left, and 12/6 o'clock represent a pure head/tailwind with no crosswind component (a wind FROM 12 is a headwind, FROM 6 a tailwind). The crosswind speed actually used by the model is:

$$W(\text{ft/s}) = -\text{windSpeed(mph)} \times 1.46667 \times \sin(\text{clockPosition}/6 \times \pi)$$

Deflection is then computed using the classic **"lag rule"**: a bullet drifts downwind by an amount proportional to how much *longer* it has actually spent in flight than a hypothetical drag-free bullet travelling in a straight line at muzzle velocity would have taken to cover the same distance —

$$\text{drift(in)} = W \times (t_{actual} - x/MV) \times 12$$

A head or tailwind component of the same wind is also modelled, as of 2026-08-14. Drag is physically a function of the bullet's velocity relative to the air, not relative to the ground, so an along-shot wind component is added to the airspeed the deceleration calculation sees:

$$\text{rangeWindFps} = \text{windSpeed(mph)} \times 1.46667 \times \cos(\text{clockPosition}/6 \times \pi)$$

This uses cos() where the crosswind term above uses sin(), so it is zero at the pure-crosswind positions (3/9 o'clock) and maximal at 12 o'clock (headwind, positive, opposing the shot) and 6 o'clock (tailwind, negative, aiding it). Inside the integrator, the relative airspeed (vx + rangeWindFps, vy) replaces ground-frame (vx, vy) only for the magnitude and direction the deceleration force is drawn from — the Mach number fed into the drag-table lookup, getCd(v/sos, tbl), uses this same relative speed. Ground-frame position and the reported velocity, energy, and Mach figures are left untouched, since those are what a chronograph or the target itself actually measures, not the bullet's airspeed.

Wind Clock convention fix (2026-08-31): the crosswind term's sign above (the leading minus on the sin() formula) was added to correct a real inconsistency. Before this fix, the crosswind term used a bare +sin() with no negation, which implemented the

opposite “destination” convention — clock = the direction the wind blows TOWARD, so entering “3” pushed the bullet right. The head/tailwind (cos) term above was already self-consistent with the “origin”/“FROM” convention (a positive value there was already documented and behaving as a headwind at 12 o’clock, i.e. wind FROM the target’s direction), so the two terms actually disagreed with each other about which convention they used. The fix only touches the crosswind (sin) term, bringing both into agreement on “clock = direction the wind is FROM,” matching how a Kestrel or any other wind meter is read, and matching the live-weather auto-fill feature elsewhere in the app, which was always deriving the Wind Clock field from Open-Meteo’s wind_direction_10m — an explicitly “degrees true, where wind comes FROM” meteorological bearing — so that feature’s crosswind output was quietly backwards until now too. The practical effect: a previously-entered clock value now produces a crosswind push on the opposite side of the clock face from before.

Magnitude: quantified against the live engine before implementation, using representative loads at their typical distances. For centrefire and rimfire the effect stays small even at 15–20mph — a few hundredths of a mil at long range, well inside the uncertainty already present from BC tolerance and chronograph error. It is proportionally larger for air rifle pellets specifically, since a given wind speed is a much larger fraction of a pellet’s much lower muzzle velocity: a light, low-BC pellet (JSB Exact .177 8.44gr, BC 0.019) at a typical 50yd Field Target distance shows roughly 0.07–0.09mil of additional drop from a 15–20mph headwind — comparable to half a scope click, and larger in angular terms than the same wind speed’s effect on the .243 Win 100gr built-in preset at 500yd. A heavier, higher-BC pellet (JSB Exact Monster .22 25.4gr, BC 0.084) shows an effect roughly ten times smaller under the same conditions, consistent with why competitive air rifle shooters favour denser, higher-BC pellets partly for reduced wind sensitivity.

Wind at Target (2026-08-31): the model above assumes a single, uniform wind for the entire flight. An optional second speed+clock reading, Wind at Target, lets the shooter describe a wind that genuinely differs at the target from the wind at the muzzle. When the field is left blank, windTargetMph is undefined and every formula below collapses back to the uniform-wind model exactly as described above.

When set, the shooter's own crosswind component W (defined above) and head/tailwind component rangeWindFps each get a matching value computed from the target reading, and the two pairs are blended by straight-line downrange fraction rather than by interpolating the raw speed-and-clock values directly:

$$\text{frac}(x) = \text{clamp}(x(yd) / \text{maxYd}, 0, 1)$$

$$W(x) = W + (W_{target} - W) \times \text{frac}(x) \qquad \text{rangeWindFps}(x) = \text{rangeWindFps} + (\text{rangeWindFps}_{target} - \text{rangeWindFps}) \times \text{frac}(x)$$

Blending the crosswind and head/tailwind components separately, rather than the shooter's raw clock positions, avoids a meaningless average when the two readings come from different directions (a 3 o'clock wind and a 9 o'clock wind do not average to "no wind" by any sensible physical reasoning; averaging their already-resolved sideways components does). rangeWindFps(x) is re-evaluated at the bullet's own current downrange position on every integration step — including inside the iterative

angle-refinement solver described in Section 4, so a zero solved at a shorter range than maxYd correctly sees only a partial blend toward the target reading, not the full one.

The lag-rule crosswind formula above, drift(in) = W × (t_actual − x/MV) × 12, only works as a single closed-form multiplication because W is constant — it factors straight out of the subtraction. Once W can vary with position that factoring is no longer valid, so the same relationship is instead accumulated incrementally, one integration step at a time:

$$dLag = \Delta t\_step - stepFt/MV \qquad driftAccum \mathrel{+}= W(x) \times dLag \times 12$$

Summed over the whole flight this telescopes back to exactly the original closed-form result whenever W(x) is constant (Wind at Target unset or equal to the shooter's own reading), since $\Sigma\Delta t\_step$ and $\Sigma stepFt/MV$ reduce to t_actual and x/MV respectively. When W(x) varies, this is the physically correct generalization rather than an approximation: dLag itself grows over the course of the flight as the bullet decelerates and spends more time covering each successive yard, so a given amount of crosswind exposure contributes more to total drift when it occurs during the slower, back half of the trajectory than during the fast, early part. Confirmed against the live engine: a crosswind confined to the first half of a 1000yd .308 Win flight (full value at the muzzle, tapering to calm at the target) produces materially less drift at the target than the same crosswind confined to the second half (calm at the muzzle, ramping up to full value at the target) — consistent with the long-range-shooting principle that a wind read near the target carries more weight than the same wind read near the firing line.

Wind at Target is a standing, session-level field like Wind Speed/Wind Clock themselves, not a property of a saved load — it is therefore excluded from preset save/load and from the load-clearing routine, exactly matching the existing treatment of the primary wind fields.

windTargetRefYd (2026-08-31 fix): frac(x) above is defined against maxYd, which several views legitimately override for their own display purposes — HUD overrides it to whatever specific distance is currently ranged, and Compare Loads' live-matched slot overrides it to the shared comparison distance. Both views clone the live params object wholesale (Object.assign against lastResults.params), so windTargetMph/windTargetClock rode along unchanged even after that fix — but frac(x) would then silently divide by the overridden maxYd instead of the range the shooter actually meant by "the target" when they entered the second reading, applying most of it far too early (or too late). windTargetRefYd is a separate field precisely to prevent that: computeTrajectory() prefers it over maxYd for frac(x)'s denominator when present, and both HUD's and Compare's live-slot params now set it explicitly to lastResults.params.maxYd — the original table's own max range — before overriding maxYd itself for their own purposes. Callers that never override maxYd in the first place (the primary calculate() path, Reticle View, non-live Compare slots) never set windTargetRefYd either, so it falls back to maxYd and nothing changes for them.

6. Atmospheric Density Correction

Drag scales with the actual air density relative to a standard reference atmosphere (15 °C, 1013.25 hPa). The density ratio is computed from user-entered altitude, temperature, humidity, and station pressure using the standard humid-air correction:

$$P_{sat} = 611.21 \times \exp\left[(18.678 - T/234.5) \times \left(T/(257.14+T)\right)\right]$$ (Arden Buck equation, T in °C)

$$P_v = (RH\% / 100) \times P_{sat}$$ (water-vapour partial pressure)

$$P_d = P - P_v$$ (dry-air partial pressure)

$$\rho_{ratio} = (P_d / P_{std}) \times (T_{std} / T_K) \times (1 - 0.378 \times P_v/P)$$

The final $(1 - 0.378 \cdot P_v/P)$ term corrects for humid air being *less* dense than dry air at the same temperature and pressure (water vapour has a lower molar mass than the nitrogen/oxygen mix it displaces) — a refinement many simplified ballistic calculators skip in favour of treating humidity as negligible.

7. Speed of Sound and Mach Number

$$a(ft/s) = 1116.45 \times \sqrt{((T°C + 273.15) / 288.15)}$$

an ideal-gas approximation referenced to standard sea-level temperature, used to convert true airspeed to Mach number for the drag-table lookup at every integration step.

8. Coriolis Effect and the Eötvös Correction

For long shots, the Earth's rotation measurably deflects the trajectory. The model uses Earth's mean angular rotation rate, $\Omega = 7.292115 \times 10^{-5}$ rad/s, combined with the shooter's latitude (φ) and the azimuth of the shot relative to true north (A), split into its two classically-distinguished components:

Eötvös effect (vertical). The bullet's motion across the rotating Earth modifies its *apparent* gravity depending on whether it is travelling with or against the direction of Earth's spin:

$$g_{eff} = g - 2\Omega \cdot \cos(\varphi) \cdot \sin(A) \times v_x$$

so an eastward-fired shot experiences very slightly reduced effective gravity (marginally less drop) and a westward shot the opposite, applied continuously through the integration rather than as an end-of-flight correction.

Horizontal Coriolis drift. A second, independent deflection is accumulated perpendicular to the line of fire, from the horizontal component of the Coriolis acceleration:

$$a_{lateral} = 2\Omega \cdot (\sin(\varphi) \cdot v_x - \cos(\varphi) \cdot \cos(A) \cdot v_y)$$

integrated twice (acceleration → velocity → displacement) alongside the main trajectory, so it correctly compounds over the whole time of flight rather than being approximated from a single average velocity.

At typical air-rifle and even most rimfire/centrefire ranges these terms are extremely small; they become practically relevant mainly at longer centrefire distances, which is why latitude and azimuth are exposed as explicit, optional inputs rather than defaulted silently.

The underlying magnitude and formula above are unchanged, but the results table presents this figure, like Wind Deflection and Spin Drift, in whichever unit — inches, cm, MOA, or mils — the table's own display toggle is set to, converted with the same

angular-conversion formulas as Section 11, rather than being fixed to linear inches or cm regardless of the table's other units.

9. Spin Drift

Spin-stabilised projectiles drift slightly to the side of the twist direction over the course of a flight, independent of any crosswind. The model uses **Litz's approximate spin-drift formula** (from *Applied Ballistics for Long Range Shooting*):

$$\text{SpinDrift(in)} = \text{twistDirection} \times 1.25 \times (Sg + 1.2) \times t^2$$

where *Sg* is the gyroscopic stability factor (Section 10) and *t* is time of flight in seconds. Being a function of t^2, spin drift grows disproportionately at long range even though it is negligible at short range — consistent with real-world long-range shooting data.

Total windage. Wherever the application presents a single figure to dial or hold, rather than the results table's three separate Wind Deflection, Spin Drift, and Coriolis columns, the three lateral corrections above are summed before rounding: TotalWindage = WindDeflection + SpinDrift + Coriolis(horizontal). All three share the same sign convention (positive drifts/pushes right), so a plain sum is correct rather than a special-cased combination — each is computed in the same angular units (MOA/mils) before being added, and the sum is converted to whichever display unit is active the same way the individual figures already are. The vertical Eötvös component is not part of this sum: it is already integrated directly into Drop (Section 8), not a separate lateral figure that would need combining in. The results table's own Wind Deflection, Spin Drift, and Coriolis columns are deliberately left as three separate figures rather than being replaced by this combined total, so a shooter can still see which of the three is contributing what; the combined total is produced only where the application hands over a single dial/hold figure for windage.

10. Gyroscopic Stability

Whether a spinning projectile is adequately stabilised is estimated with **Miller's stability formula**:

$$Sg = 30 \cdot W / (n^2 \cdot d^3 \cdot l\_c \cdot (1 + l\_c^2) \cdot \rho\_ratio)$$

where *W* is projectile weight in grains, *d* is diameter in inches, *n* is the twist rate expressed in calibers (twist length ÷ diameter), and *l_c* is the length-to-diameter ratio. The same relationship is solved in reverse to report the minimum twist rate required for a given projectile:

$$T\_min = d \times \sqrt{(30 \cdot W \cdot \rho\_ratio / (1.5 \cdot d^3 \cdot l\_c \cdot (1 + l\_c^2)))}$$

Greenhill's rule is also available as a simpler, independent cross-check:

$$T = C \times d^2 / l \qquad (C = 150 \text{ below 2700 fps muzzle velocity, } 180 \text{ at or above it})$$

Shape correction factors. Miller's formula implicitly assumes a roughly solid, uniform-density projectile, so it has no way to account for a boat-tail shedding mass toward the rear, or a diabolo pellet's mostly-hollow skirt — both of which need less spin than the formula would otherwise predict for the same length-to-diameter ratio. The application applies commonly-cited, community-sourced correction factors on top of the raw Miller

result — explicitly documented in the source as approximate corrections, not a first-principles recalculation of the projectile's actual moment of inertia:

Shape	Sg multiplier	Twist-requirement multiplier
Flat-base	×1.00	×1.00
Boat-tail	×1.08	×√1.08
Diabolo (airgun pellet)	×1.20	×√1.20

The resulting Sg is classified into bands for display: below 1.0 *Unstable*, 1.0–1.4 *Marginal*, 1.4–2.5 *Stable*, 2.5–4.0 *Over-stabilised*, above 4.0 *Excessively fast*.

A known, documented gap: Miller's base formula (and the shape factors above) is referenced to roughly 2800 fps rifle velocities, while air-rifle loads typically run 700–950 fps. Miller himself did provide a velocity-correction factor, (V/2800)^(1/3), and capped it at its value at the speed of sound rather than letting it shrink further as velocity drops below Mach 1 — so a naive application of the raw exponent at airgun velocities is not what Miller intended, and a correctly-floored correction would move Sg by a meaningful margin (roughly ×0.74 rather than ×0.63–0.70 across the 700–950 fps range). The application does not currently apply this correction. That has been deliberately deferred rather than added on the strength of Miller having written it down: his formula and its velocity correction were both derived from and validated against conventional supersonic rifle bullets, not 700–950 fps diabolo or slug-shaped airgun projectiles, so there is no clear evidence that adding even the correctly-floored correction would improve — rather than simply relabel — the app's current Sg estimate for that regime.

11. Energy and Angular-Unit Conversions

Kinetic energy at any point in the flight:

$$KE(\text{ft·lb}) = v^2 \times (W_{gr} / 7000) / (2 \times 32.174)$$
$$KE(\text{J}) = KE(\text{ft·lb}) \times 1.35582$$

using 7000 grains per pound and g = 32.174 ft/s² as the weight-to-mass conversion, and the standard ft·lb-to-joule constant.

Linear drop and drift are converted to angular holdover units using the standard small-angle approximations, with a $\cos^2$(shot angle) correction applied to project the true (sloped) range onto the horizontal — the same simplification commonly known as "rifleman's rule":

$$\text{MOA} = -\text{drop(in)} \times \cos^2(\text{angle}) / (\text{range(yd)}/100 \times 1.0472)$$
$$\text{Mils} = -\text{drop(in)} \times \cos^2(\text{angle}) / (\text{range(yd)} \times 0.036)$$

(1.0472 in per 100 yd defines one MOA; 0.036 in per yd defines one mil — both standard reference constants.)

12. Trajectory Truing (Empirical BC Calibration)

Manufacturer-quoted BCs are averages and frequently do not match a specific rifle/load/chronograph combination exactly. Trajectory Truing lets a shooter correct

for this using one real, observed data point: a known range and the holdover (in mils, MOA, inches, or cm) they actually needed to hold at that range.

The engine solves for a single **BC multiplier** by binary search over the range [0.3, 2.5] (up to 40 iterations, convergence tolerance 0.0005 units in whichever unit the observation was entered in):

- A higher BC means less drag and therefore less drop, so a smaller predicted holdover value.
- If the model currently predicts *more* holdover than observed, BC is too low and is nudged up; if it predicts *less*, BC is too high and is nudged down.
- If the observation falls outside what any BC in the search range could produce, the multiplier is clamped to whichever bound gets closest, rather than continuing to search fruitlessly.

The resulting multiplier is applied to the base BC for the remainder of that session (and can optionally be saved back into the load's preset), effectively calibrating "how much this specific bullet actually decelerates" from one field observation rather than trusting the catalogue BC in isolation.

A separate, deliberately non-destructive **Verification** step lets the shooter check that calibration against a *second*, independent real observation at a different distance, without re-fitting anything. It reports the model's prediction, the observed value, the difference, and a qualitative verdict (holding up well / some drift / diverging), the last of which is framed as a prompt to re-check muzzle velocity or another input — since a genuinely good BC fit should generalise across distances, and a growing gap downrange usually points to a different input being wrong rather than the BC itself.

13. Zero Offset (Manual Holdover Correction) — Elevation and Windage

A shooter who has zeroed a rifle for one load, then switches to a different projectile without re-zeroing, often already knows from field experience how much that new load holds over or under point of aim at the zero range. Zero Offset lets that known figure be entered directly — in mils, MOA, or clicks — and applied to the whole table, without touching the drag model at all. This correction was originally elevation-only; a second, independent field applies the identical idea to windage, covered later in this section.

Both the elevation and windage forms of Zero Offset are deliberately distinct corrections from Trajectory Truing above, not variants of it. Truing solves for a BC multiplier so the drag model matches one observed data point, which implicitly assumes the zero itself is a true, zero-drop reference. Zero Offset does the opposite: it takes the zero as a starting point that is deliberately not a true zero, and shifts the relevant table column by a constant angular amount to match a directly known holdover.

The elevation offset is converted once, on entry, to a single canonical mils-equivalent value, then applied as a term added to the drop calculation at every range:

$$\text{drop(in)} \mathrel{-}= \text{offsetMils} \times \text{range(yd)} \times 0.036 / \cos^2(\text{angle})$$

Because mils and MOA are angular units, a constant angular holdover corresponds to a growing linear correction downrange rather than a fixed number of inches — the term

above scales with range for exactly that reason, and divides through $\cos^2$(angle) so an inclined shot's holdover is not distorted by the same slope correction already applied to drop (Section 11). Since the correction modifies the shared drop value every other output is derived from, the results table, chart, CSV export, Range Card view, and turret-strip graphics all reflect it automatically with no separate implementation in each; energy, velocity, windage, and spin-drift columns are untouched, since the correction is a pure holdover shift rather than a change to the physical model.

A second, independent field — Windage Zero Offset — applies the identical idea to the windage column instead of drop, for a known left/right correction at the zero range (for example a scope that is not perfectly boresighted, or a previous zero taken in a crosswind). It has its own mils/MOA/clicks unit toggle, entirely separate from the elevation field's, and is converted to a canonical mils-equivalent the same way before being applied:

$$\text{windage(in)} \mathrel{+}= \text{offsetMils} \times \text{range(yd)} \times 0.036 / \cos^2(\text{angle})$$

Two details distinguish this from the elevation term above, both a direct consequence of what the drop and windage values represent structurally. First, the sign: the drop formula above subtracts, because the downstream elevation formulas (moa, mils) already carry a leading negative sign relative to drop, so subtracting there is what makes a positive entered value read as a positive hold-up figure. The windage formulas carry no such leading negative sign, so the windage term above adds rather than subtracts — a positive entered value comes out as a positive, right-hand correction, matching the sign convention used throughout the rest of the app for Wind Deflection, Spin Drift, and Coriolis (positive = right). Second, unlike drop, windage is not already exactly zero at the zero range by construction — real crosswind deflection at a typical zero distance is normally small but not zero — so this term is simply "shift the whole windage figure by a known constant amount," not "make the zero row read a real value instead of always zero" the way the elevation version is. The offset is applied to the same shared windage value that Wind Deflection, Spin Drift, and the horizontal Coriolis component are summed into (Section 9), before that sum is rounded — so it appears in the main results table's Wind column, and in the combined windage figure Reticle View, HUD, and the Turret Dial's Windage control use (see the combined-windage design note at the end of Section 9). Elevation and windage offsets are read from independent fields and applied to independent variables, so setting one has no effect on the other.

Both fields also accept a third entry unit, clicks, alongside mils and MOA. Unlike mils and MOA — fixed ratios to each other that never change — a click's angular size depends on the turret it represents, so clicks resolves through the same clickSizeMoa global already shared by the results table's own clicks column and the Turret Dial (Part 3, Section 3), rather than being a third independently fixed constant: raw = clicks × clickSizeMoa (a MOA-equivalent, so this feeds directly into the same ×1.0472/3.6 conversion used for a direct MOA entry). Selecting clicks on either field reveals the same click-size selector used everywhere else in the app; choosing a size there updates clickSizeMoa globally, so the other Zero Offset field's selector (if also showing) and the results table's own selector all stay in agreement, since there is only ever one current click size for the whole app. This is a deliberate extension of the existing "clicks are always live, never baked" principle into an editable field rather than a read-only one: what is actually saved to a preset is the raw click count and the unit string "clicks," not a frozen angle, so if the click-size selector is changed after saving, the same typed click count resolves to a

different angle the next time Calculate runs — consistent with how the results table's own clicks column already behaves, not a new class of behaviour introduced here.

Which unit the Zero Offset field defaults to was originally tied to calibre — the same convention the results table itself has always used, where .177 loads default to MOA and everything else defaults to mils. A user report showed that heuristic to be the wrong signal for Zero Offset specifically: calibre says nothing about which turret is actually mounted on a given rifle, and a .308 with an MOA-turret scope or a .177 with a mil-dot scope are both real, common configurations. The fix replaces the calibre guess with a per-preset stored preference — a new angularUnit field, captured automatically the first time a preset is saved with the results table showing mils or MOA, and preferred over the calibre guess on every later load of that same preset. Saving while viewing a non-angular display mode (clicks, inches, or centimetres) neither sets nor overwrites this preference, so briefly checking click values before saving can never silently erase a preference set earlier; a preset that has never been saved on mils or MOA at all simply falls back to the original calibre-based guess, exactly as before this field existed, which keeps every preset saved prior to this change behaving identically. Both Zero Offset toggles then default to whichever unit that resolution settles on, rather than always mils.

14. Plausibility Checks

Before every calculation, entered values are checked against soft, per-discipline bounds (air rifle / rimfire / centrefire) on muzzle velocity, projectile weight, and BC (0.01–1.0 for all disciplines, the realistic range for small-arms projectiles generally). These checks are deliberately **advisory only** — they surface a dismissible warning (e.g. "muzzle velocity looks unusual — check fps vs m/s") to catch likely typos or unit mix-ups, but never block or alter a calculation. This reflects a considered design choice, discussed further in Part 2.

Part 2 — Design Methodology

1. Single-File, Zero-Dependency Architecture

The entire application — markup, styling, and logic — is one self-contained HTML file with no build step, no server component, and no external runtime dependencies. This was a deliberate, foundational constraint rather than an accident of scope: it guarantees the calculator works fully offline (relevant in the field, away from signal), opens instantly in any browser including Safari on macOS and iOS, and can be copied, backed up, or shared as a single file with no installation process. Every feature added over the project's life had to fit within that constraint, which in turn shaped several other decisions below.

This constraint was tested directly when the user asked whether the Range Card could integrate with an Apple Watch. watchOS has no general-purpose web browser, so there is no way for a device to load the application directly, and building a companion watch app or a backend sync service would have meant abandoning the single-file, server-free model rather than working within it. The resolution kept the constraint intact: a "Share"

button was added to the Range Card using the standard Web Share API, handing the currently displayed range/holdover data to the native iOS/macOS share sheet as plain text, which the user can then route to Notes, Reminders, or a Shortcut — several of which already sync to and display on a paired Watch without the application needing to know anything about watchOS at all. A clipboard-copy fallback covers browsers without share-sheet support, so the button degrades gracefully rather than failing silently. A follow-up report showed one concrete cost of that architectural choice: on macOS, sharing to Notes produced an empty note with only a link back to the application instead of the range text, because Safari's share sheet — invoked from a regular browser tab rather than an installed web app — also offers the current page itself as a shareable item, and Notes' share extension preferred that page-link item over the supplied text when both a title and text were present with no url. This is a documented WebKit behaviour, not something the application controls; the mitigation was to stop supplying a title and share text alone, the combination most consistently reported to avoid the substitution, with the pre-existing clipboard fallback remaining available if a target app still prefers a link card.

A later refinement extended the Range Card with a wind-correction reference, prompted by a design question about whether wind should be part of a printed range card at all. Unlike elevation holdover, which is deterministic for a given rifle/load/zero and therefore safe to state as a single static number, wind correction on this application depends on a single wind-speed-and-direction input entered before Calculate — one static guess standing in for real field wind, which varies continuously. Printing that single computed correction directly on the card risked implying a precision the input never had. The resolution mirrors how printed dope/wind cards have traditionally handled this: rather than a baked-in correction, each range row also shows a per-1mph (or per-1kph, in metric mode) reference figure — drift divided back down by the wind speed used in the calculation, expressed in kilometres per hour rather than miles per hour whenever the page is set to metric, matching how every other Range Card value already follows that toggle — in whichever unit — mils, MOA, clicks, inches, or centimetres — the elevation value is already shown in, so the shooter can scale it by whatever wind they actually observe at the line rather than trusting a single number computed from a guess. The figure is labelled explicitly as "Wind (per mph)" or "Wind (per kph)" rather than a bare unit suffix, since a follow-up review noted that a plain "(mph)" or "(kph)" reads like a wind-speed reading rather than a rate — easy to misread as "wind is 0.28mph" instead of "0.28 for every 1mph of wind." Angular units (mils, MOA) are shown as decimals, consistent with how drift is displayed everywhere else in the application; clicks are rounded to a whole number, since clicks are inherently discrete. The figure is omitted entirely when the calculation used 0mph wind, since there is no rate to derive. The addition required no change to the underlying physics or the plain-text Share export path, since both already derive their values from the same per-row drift fields the main results table uses; the wind reference simply reads them a second time and divides.

2. Canonical Internal Units, Converted Only at the Boundary

All physics computation is carried out in imperial units internally, regardless of whether the user has the interface set to imperial or metric display. Metric values are converted to imperial on input and back to metric only for display. This was chosen over maintaining two parallel unit-aware code paths through the physics engine, which

would have doubled the surface area for bugs and made the two systems drift out of agreement over time.

This decision was validated by a real bug it prevented from being worse than it was: an early metric-interval implementation rounded each successive table row's *native* interval mark (e.g. every 10 m) to the nearest yard independently by repeatedly adding a fixed yard increment, which let small rounding errors compound row over row into a visible drift (…80 m, 91 m, 101 m, 111 m…). The fix — generating each target mark directly in its native unit and converting *each one independently* to the nearest yard, rather than accumulating an already-rounded step — is a direct consequence of the "single canonical unit, convert at the boundary" philosophy: the bug was in the conversion boundary, not the physics, and was fixed there without touching the engine at all.

3. Iterative, Usage-Driven Feature Development

Features were added in direct response to concrete friction encountered in real use, rather than speculative up-front design. Trajectory Truing and its later Verification step, the Range Card view, live-weather auto-fill, preset drag-reordering (including a dedicated touch-based reimplementation once it became clear the native HTML5 drag API silently does nothing for ordinary elements on iOS Safari), and the recent overflow-menu consolidation of per-preset actions are all examples of a feature being introduced, used, found lacking in some specific way, and then refined — sometimes across several successive passes — rather than being designed exhaustively before first use.

Compare Loads is the most extensive recent example of this pattern: shipped first with a fixed, auto-computed reference distance and an explicit Compare button, then made editable on request, then extended to read a currently-loaded preset's live, unsaved amendments rather than always the stored version, and finally refined again once real use showed one of its warnings was firing too often to stay useful. See Part 3.

4. Testing Without a Browser: Headless Node/VM Harnesses

The development environment has no access to a real browser or to the npm package registry, which rules out both conventional browser-automation testing and most off-the-shelf JavaScript testing frameworks. In their place, a custom pattern was developed: the application's actual <script> block is loaded verbatim into a Node vm context, against a small hand-built fake DOM (fake elements supporting classList, dataset, closest(), event listeners, and enough of the canvas 2D context API to record what a chart would have drawn) and a fake localStorage. This allows the *real, unmodified* production functions — computeTrajectory, renderPresets, drawChart, and so on — to be exercised directly and their outputs asserted on, without reimplementing or mocking the logic under test.

Every fix or new feature is paired with a test that specifically exercises the bug or behaviour in question — not a superficial smoke test — and existing harnesses are re-run after every change to catch regressions. Several bugs were caught specifically because a harness was built to reproduce the *exact* reported symptom (for example, simulating a touch landing on a nested SVG icon rather than its parent button, which a naive same-element check would miss) rather than a generic version of the problem.

5. Multi-Variant Deployment Discipline

The calculator exists as four synchronised files: the full-featured main application, a stripped-down “General Release” public build with no built-in preset library, and two deployment mirror copies (a site-root index.html and a personal-folder copy) kept byte-identical to their source in every shared code path. Every change to shared logic — the preset system, the chart, the physics engine — is applied identically across all four and reverified in each before being considered complete, rather than treating one file as canonical and letting the others drift.

6. Transparency About Approximations

Several of the physics relationships used are well-established approximations rather than exact derivations, and this is treated as something to document plainly rather than obscure behind confident-looking output. The G1↔G7 BC estimate ratio, the shape-based stability correction factors, and the community-sourced airgun drag table are all flagged in code comments as approximations with a stated provenance and stated limits on where they’re valid — and the project maintains an explicit, honestly-worded “known gaps” list (for example, the missing muzzle-velocity correction term in the stability calculation) rather than presenting the current model as complete.

7. Advisory Validation, Not Gatekeeping

Where the application checks user input for plausibility — velocity, weight, BC — it always does so as a dismissible, non-blocking warning rather than a hard validation error. The design preference throughout is to help a user catch their own likely mistake (a decimal point, an fps/m-s mix-up) without ever preventing them from running a calculation on genuinely unusual but real inputs, such as an intentionally low-powered air rifle load or an experimental wildcat cartridge.

8. Accessibility as a First-Class Concern

Distinguishing information is never conveyed by colour alone. On the trajectory chart, the drop and energy curves are differentiated by both colour *and* line style (solid versus dashed) so the chart remains legible to colourblind users; the same principle was applied when auditing other colour-only status indicators elsewhere in the interface. This was addressed as a dedicated pass over the application rather than an incidental side-effect of a colour palette choice.

9. Deliberate Simplicity Over Feature Completeness

Not every capability that was built was kept. A Bluetooth integration with a Kestrel 5700X weather meter was implemented and then deliberately removed once two problems became clear: Web Bluetooth is Chrome-only, directly conflicting with the project’s core Safari/iOS compatibility requirement, and the meter’s proprietary wind-speed encoding could not be reliably decoded. Rather than persisting with a partially-working, platform-incompatible feature, it was replaced with a simpler browser-geolocation-plus-weather-API “live weather” button that works identically everywhere the application itself runs. This stands as the clearest example of the project consistently preferring a smaller feature that works everywhere over a larger one that only works some of the time.

10. The Changelog as an Engineering Discipline

Every change, however small, is recorded in a dated changelog entry written in prose: what changed, why it was needed, and how it was verified. This is treated as part of the engineering process itself rather than after-the-fact documentation — it is what makes the "transparency about approximations" and "usage-driven iteration" principles above actually auditable after the fact, rather than asserted.

Part 3 — The Compare Loads Feature

1. Purpose and Architecture

Compare Loads lets a shooter select up to three saved presets — any mix of built-in and custom, and any mix of the three disciplines — and see headline stats and an overlaid trajectory chart for all of them side by side, motivated directly by a real use case: deciding which of several rifle/load combinations to bring to a competition.

The feature required no changes to the trajectory engine itself. Because computeTrajectory() is a pure function of a self-contained params object with no dependency on the live form, a params object can be assembled for any saved preset and passed through the same, unmodified engine used for the single-load Results page. Comparison is therefore an assembly-and-presentation problem — building the right params object for each selected load and rendering three results side by side — rather than a second calculation path that risks drifting out of agreement with the first.

2. Reference Distance Resolution

All compared loads are shown out to one shared distance, so their chart lines end together and every headline figure is read at the same range. By default this is the shortest of the selected loads' own max ranges — the furthest distance every selected load can be shown at without extrapolating any of them — but the distance is also directly editable, defaulting to that auto-suggested figure and re-suggesting it as the selection changes, until the shooter types a value of their own, at which point the typed value is left alone across further selection changes and only resets to auto-suggest once the selection is cleared back to zero.

Recomputation is explicit rather than live: neither changing the selection nor editing the distance field recalculates anything by itself, only the next press of the Compare button — consistent with the single-load page's own Calculate Trajectory model, and avoiding a rapid sequence of recomputations while a shooter is still deciding what to compare.

The distance is not capped to any load's own max range; the engine will extrapolate to whatever value is typed in. Instead, a load shown beyond its own saved (or, for a live-matched load, currently configured — see Section 4) max range triggers a dismissible, non-blocking warning rather than a hard limit, consistent with the advisory-only validation philosophy established in Part 2, Section 7.

3. Display Unit Selection (Drop & Wind Drift)

The Drop and Wind Drift rows of the comparison table can be read in mils, MOA, inches, or cm, chosen from a dropdown rendered directly above the table — independent of whichever unit the single-load Results table happens to be showing, since a shooter comparing loads may reasonably want a different readout in each place.

The selection is held in a dedicated compareUnitDrop state variable, defaulting to mils to match the single-load table's own default, and switching it is a pure re-render rather than a recomputation: every unit variant — dropIn, dropCm, moa, and mils for Drop; driftIn, driftCm, driftMoa, and driftMils for Wind Drift — is already present on each result row from the original calculation, exactly as the single-load table's own five-way unit toggle already relies on. runCompare() caches its last-computed list and reference distance; setCompareDropUnit() simply re-renders from that cache. The row labels update to state the active unit explicitly (e.g. "Drop (mil) at 500yd"), removing an earlier ambiguity where the same two rows were fixed to whichever of inches or cm matched the Imperial/Metric toggle, with no unit named in the label at all.

A "clicks" option was added to this dropdown to match the single-load table, once that table gained its own Wind Drift, Spin Drift, and Coriolis clicks support. The two click-size selectors are intentionally not independent: both single-load and Compare read from and write to the one shared clickSizeMoa global, so changing the click size in either view updates the other, and there is only ever one "current" click size for the whole app rather than two that could silently disagree. Selecting clicks reveals the same click-size selector already used elsewhere, reusing the identical dropdown markup rather than a separate implementation.

4. Live vs. Saved State Resolution

A selected preset may also be the one currently open on the single-load Inputs tab, possibly with amendments that have not yet been saved back to the preset — a BC from Trajectory Truing, an edited Zero Offset, a changed weight. Compare resolves this by identity rather than by trying to detect whether something has changed: a selected slot is treated as "live" only when its mode, preset type, and name all match the preset currently loaded on the Inputs tab and a calculation has actually been run for it — the same name-based identity match already used elsewhere in the application for the baked-truing badge.

When a slot matches, Compare clones the exact params object already driving the live Results page — rather than rebuilding one from the stored preset — so any unsaved amendment is picked up automatically, with the shared reference distance substituted in. The clone is never written back to either the live form state or the saved preset; Compare only ever reads. When a slot does not match — a different preset selected, nothing loaded yet, or the mode has since been switched — it falls back to building a fresh params object from the stored preset, exactly as it would if the live-matching logic did not exist at all. No separate "has this been amended" detection was needed: reading live state whenever the identity matches produces identical figures to the saved preset whenever nothing has actually changed, so the fallback behaviour falls out automatically rather than being a distinct code path.

Condition	Params source	Shown as

Selected preset is loaded on Inputs, with a calculation already run	Cloned from the live results (lastResults.params)	"(live)"
Selected preset is not currently loaded, or nothing has been calculated yet	Rebuilt fresh from the stored preset	Preset name only
Mode has been switched since a preset was loaded	Rebuilt fresh from the stored preset (live-match state is cleared on mode switch)	Preset name only

5. Wind Speed and Direction Override

Wind is the one environmental condition Compare lets a shooter override independently of the single-load Inputs tab, added after user feedback asking whether wind drift could be adjusted the same way the comparison distance already can be. The feature reuses that exact same auto-suggest/customize/reset pattern (compareDistanceCustomized / fCompareRange) rather than inventing a new one: a speed field and a direction dial both default to whatever is currently set on the live Wind Speed and Wind Clock fields and keep re-mirroring them on every selection change, right up until the shooter touches either control, at which point both latch to the override and stop re-syncing. A single shared latch (compareWindCustomized) covers both fields rather than one each, since speed and direction are really one concept for this purpose — "the wind used for this comparison" — and a shooter who has already overridden the speed almost certainly wants their own direction too, not to have it silently snap back to today's live value the next time a load is added or removed from the selection.

Direction is set with a 72px drag-anywhere clock face rather than twelve discrete numbered buttons. At any size that fits inline with the rest of the comparison controls, twelve separate tap targets around a small ring would each cover only a few pixels of circumference — well under a reliable touch target on a phone — so accuracy would depend on hitting a specific small spot rather than a general area. Instead the whole ring is one continuous drag/tap surface: clockHourFromAngle() takes a pointer offset from the dial's center and returns the nearest hour via atan2, using the same 12-up/3-right/6-down/9-left mapping already established by the Wind Clock field, so the same "clock = direction the wind is FROM" convention holds everywhere in the app. Pointer Events (rather than separate mouse and touch listeners) drive the interaction, with setPointerCapture keeping a drag tracked even once a finger slides off the small circle, which a fast or wide drag on a touchscreen would otherwise lose the moment the finger crossed the dial's edge.

Wiring the override in required one correction beyond the obvious change to buildParamsFromPreset(): the live-matched slot described in Section 4 above does not call buildParamsFromPreset() at all — it clones lastResults.params directly, which carries whatever wind was in effect the last time Calculate ran on the Inputs tab. Left alone, that would mean the override worked for every compared load except the one most likely to already be on screen: the load currently loaded on the Inputs tab. The isLive branch in runCompare() therefore re-injects windMph and windClock into that clone explicitly, alongside the maxYd override it already applied for the comparison distance, so both paths agree on which wind is actually being used.

6. Per-Slot Velocity Override

An earlier feature added a per-slot velocity override to Reticle View, letting a shooter recompute a single reticle's hold under a hypothetical muzzle velocity. A follow-on request asked for the same capability on Compare Loads, but with a deliberate difference in scope: rather than one shared override applied to every compared load at once (the pattern already used for wind and the comparison distance), velocity is overridable independently per column, so a shooter comparing three loads can nudge just one — for example, testing a chronograph reading against a single load's book value — without disturbing the other two. This was a deliberate design choice, not an oversight: wind and distance are properties of the comparison as a whole ("what conditions are we comparing under"), while a load's velocity is a property of that specific load.

State for this feature is a single object, compareVelocityOverrides, keyed by compareRefKey(ref) (a "mode:type:idx" string) rather than by the load's position in the results table. This distinction matters because column order follows compareSelection, which can reshuffle as slots are toggled off and on — keying by ref identity rather than column position means an override stays attached to the load it was set on, and can never silently reattach itself to a different load that happens to land in the same column afterward. The control itself lives inside each column's own header cell in the results table, rather than alongside the shared Wind/Compare-to controls above the picker, since it is genuinely per-column state and reads more clearly attached to the load it affects.

buildParamsFromPreset() gained a fourth parameter, forcedMvFps, changing mvFps:p.mv to mvFps:forcedMvFps||p.mv — the same optional-override pattern already used for forcedMaxYd. The live-matched slot in runCompare() needed the same re-injection fix already applied for the wind override in Section 5 above: because it clones lastResults.params directly rather than calling buildParamsFromPreset(), it would otherwise silently ignore its own velocity override while every other slot correctly picked theirs up. The clone now includes mvFps:compareVelocityOverrides[compareRefKey(x.ref)]||lastResults.params.mvFps, falling back to the velocity in effect when Calculate last ran rather than a nonsensical unset value.

The per-column input is button-gated, consistent with how the Range and Wind fields already behave: typing a value stores it in compareVelocityOverrides without recomputing anything, and it only takes effect the next time the Compare button is pressed. The per-slot Reset link that appears once a column has an override is the one exception — it takes effect immediately, deleting that slot's entry and re-running the comparison right away, rather than leaving the shooter to wonder why "Reset" appeared to do nothing until they pressed Compare again. A deselected slot's override is discarded in toggleCompareSelect()'s removal branch, since it is tied to that specific load rather than to whichever load might occupy that slot next; clearing the selection back to zero clears every override at once, alongside the existing Range and Wind latch resets.

7. Centrefire-Scoped Subsonic-Crossing Warning

An earlier version of Compare flagged any load that crossed into subsonic flight before the comparison distance, using the same per-row isSubsonic flag the single-load Results table already computes. In practice this fired on a large share of comparisons in the air rifle and rimfire disciplines, where subsonic flight is the ordinary case — airgun pellets and slugs, and standard-velocity rimfire ammunition, are subsonic as a matter of course — making the warning noise rather than signal for exactly the disciplines where it fired most often.

The warning is now gated to centrefire selections only. Subsonic flight in a centrefire load is usually a deliberate choice rather than the default — a purpose-built subsonic .300 BLK load for a suppressed rifle is the clearest example — so a warning there is more likely to be catching something genuinely worth a second look. The gate is a single mode check against the selected preset's own recorded discipline, evaluated per selected load rather than globally, so a mixed comparison of a centrefire and an air-rifle load correctly warns for one and stays silent for the other in the same comparison. This mirrors the project's wider preference, discussed in Part 2, Sections 6 and 7, for validation that reflects what is actually unusual for the discipline in question rather than a single blanket rule applied everywhere.

8. Testing Methodology

Compare Loads is exercised with the same headless Node/vm harness pattern described in Part 2, Section 4, extended to simulate full real-world interaction sequences rather than only individual helper functions — selecting a preset, editing a field, calculating, opening Compare, and pressing Compare again, in that exact order — since the live-matching logic in particular depends on state built up across several successive user actions rather than any single function call.

One deployment variant, the General Release build, intentionally ships with no built-in preset library, which meant the comparison test suite could not assume any built-in preset indices exist. Tests were written to seed their own custom presets directly, via the same save path a real user would go through, making the suite portable across all four synchronised files regardless of which one, if any, ships built-in data — consistent with the multi-variant deployment discipline described in Part 2, Section 5.

Appendix — Key Constants Reference

Constant	Value	Used for
g (gravity)	32.174 ft/s^2	Trajectory integration, energy calculation
Ω (Earth's rotation rate)	7.292115×10^{-5} rad/s	Coriolis / Eötvös effect
Standard temperature	288.15 K (15 °C)	Air density ratio, speed of sound
Standard pressure	101325 Pa (1013.25 hPa)	Air density ratio

BC standard ratio (G1→G7 estimate)	1.9	Estimating BC(G7) when not entered
Spin drift coefficient	1.25	Litz spin-drift formula
Shape factor — flat-base	×1.00	Stability (Sg) correction
Shape factor — boat-tail	×1.08	Stability (Sg) correction
Shape factor — diabolo pellet	×1.20	Stability (Sg) correction
Greenhill constant (MV < 2700 fps)	150	Alternative twist-rate rule
Greenhill constant (MV ≥ 2700 fps)	180	Alternative twist-rate rule
1 MOA	1.0472 in @ 100 yd	Angular holdover conversion
1 mil	0.036 in @ 1 yd (3.6 in @ 100 yd)	Angular holdover conversion
ft·lb → J	× 1.35582	Energy display conversion
Grains per pound	7000	Energy calculation mass conversion
Integration step	3 ft (1 yd)	RK4 trajectory integration
Trajectory Truing search range	BC multiplier 0.3–2.5	Empirical BC calibration